



BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing



Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

A – Actuators

S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width, self.height, self.food_positions, self.opponents, self.walls, self.agent_pos`

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent system. The `self.opponents` is an array that initializes multiple opponent entities that actively take their own turns and move around the grid alongside the main agent, rather than just being static objects.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

Percept sequence is the complete history of everything an AI agent has sensed or observed from the moment it started until now

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` method represents the Sensors component of the PEAS framework. It acts as the agent's sensory input by gathering and returning information about its surroundings, such as its current position, the locations of opponents etc

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This is a Partially Observable environment. The agent does not have a map in its mind of where every wall or piece of food is located. It can only sense the food or walls when it steps directly on top of them

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure will be updated because the `self.score` gets -5 points deducted if it hit a wall

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because a Performance Measure must judge the actual results the agent achieves, not just how hard it worked to get there. If we scored the agent's internal processing (its 'busy work'), the agent might just waste time doing useless calculations to get points. By only scoring changes in the external environment (like hitting a wall), we

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It does not change the overall classification, because the environment is already Partially Observable. However, it alters the environment by introducing a completely unobservable hazard. Since the trap data is hidden, the agent cannot detect the traps at all, making the environment even more blind for the agent.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the 'smells_toxin' sensor helps the agent act with Rationality because it allows the agent to identify toxin and get action.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

points, the robot learns to clean up the dirt, dump it back onto the floor, and clean the exact same spot over and over again. This illustrates the danger of faulty metrics: we must score the actual result (having a clean room) rather than the internal process (the act of vacuuming)

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING